



Testes e Qualidade de Software

Simulador de Batalha – PokeSal

Guilherme Monteiro Da Silva ^{*1}, Miguel Novaes Palma Santos ^{†1}, Dantton Lavigne ^{‡1}, Guilherme Gomes de Magalhães ^{§1}, Pedro Arthur de Melo Nascimento ^{¶1*}

¹ Engenharia de Software

Escola de Tecnologias

Universidade Católica do Salvador (UCSAL)

Av. Prof. Pinto de Aguiar, 2589 Pituaçu, CEP: 41740-090

Salvador/BA, Brasil

Setembro de 2026

Resumo

Relatório do PokeSal, na matéria de Testes e Qualidade de Software. O jogo é uma batalha 1 contra 1 no estacionamento da UCSal: cada um escolhe um inicial, o terreno mexe no dano e a luta vai até o fim. Aqui entram a leitura do enunciado, os três requisitos autorais, os diagramas, o que fizemos no código para seguir o Google Java Style, os bugs que apareceram no caminho e o trabalho de cada um, checklist estático do código, testes JUnit, SonarQube (bugs, vulnerabilidades, code smell e coverage) e quem fez o quê nessa etapa.

Palavras-chaves: PokeSal. Requisitos. UML. Checkstyle. JUnit. SonarQube.

^{*} guilhermemonteiro.silva@ucsal.edu.br

[†] miguelnoaves.santos@ucsal.edu.br

[‡] dantton.pinheiro@ucsal.edu.br

[§] guilherme.magalhaes@ucsal.edu.br

[¶] pedroam.nascimento@pro.ucsal.br

Sumário

Sumário	2
1 Introdução	3
2 Análise estática de requisitos	3
2.1 O que já vinha pronto	3
2.2 O que o texto deixou em aberto	3
3 Requisitos autorais	4
3.1 RA01 – Combo	4
3.2 RA02 – Lutar até o fim	4
3.3 RA03 – Um status por vez	4
4 Modelagem UML	4
4.1 Diagrama de casos de uso	5
4.2 Diagrama de classes	5
5 Implementação e boas práticas (Checkstyle)	7
6 Problemas encontrados e correções	7
7 Relatório de contribuição individual	8
7.1 Guilherme Monteiro Da Silva	8
7.2 Guilherme Gomes de Magalhães	8
7.3 Miguel Novaes Palma Santos	8
7.4 Dantton Lavigne	8
7.5 O que a gente fez junto	8
8 Uso de inteligência artificial	8
9 Fase 02 – Qualidade, testes e SonarQube	9
9.1 Checklist de teste estático (revisão manual)	9
9.2 Testes JUnit e matriz de rastreabilidade	10
9.3 SonarQube	10
9.4 Bugs da Fase 02 e o que a gente corrigiu	12
9.5 Contribuição individual da Fase 02	12
10 Considerações finais	13

1 Introdução

O trabalho é um torneio de batalhas no estacionamento da UCSal Pituaçu. Cada treinador entra com um inicial (BulbaSal, CharSal, SquirtSal, ChikoSal, CyndaSal ou TotoSal). A luta é por turno, com tipo, terreno, status e no máximo dois itens.

Na Fase 01, entregamos a análise dos requisitos, três regras autorais, UML (casos de uso e classes), código no Git e um relatório dizendo o que cada um fez.

Na Fase 02, a entrega é qualidade: revisão manual do código, JUnit com os nomes que o enunciado pede (e os autorais), análise no SonarQube e o registro do que a gente achou e corrigiu.

2 Análise estática de requisitos

Lemos o enunciado antes de sair programando, para marcar o que estava claro, o que estava solto e o que a gente ia ter que decidir.

2.1 O que já vinha pronto

- um inicial só, entre os seis nomes;
- HP, ATK, DEF, SPD e tipo;
- Fogo, Água e Planta (2,0x e 0,5x);
- Asfalto Quente: Fogo +15% de dano;
- Poça de Chuva: Golpes de Água aplicam 10% adicionais de precisão ou dano;
- Canteiro Central: Planta recupera 5% do HP máximo no fim do turno;
- ordem do turno pela SPD;
- status no fim do turno (Queimado, Envenenado, Paralisado);
- no máximo dois itens, e item gasta o turno.

2.2 O que o texto deixou em aberto

A poça de chuva dá “10% a mais de precisão ou dano” para Água. Precisão e dano não são a mesma coisa. Se a gente colocasse chance de errar, cada teste dava um resultado diferente. Ficamos com **dano**.

O enunciado cita os status, mas não diz quem aplica. Não tem item de status nem sorteio. A gente ligou no golpe elemental: Fogo queima, Água paralisa, Planta envenena.

Queimado “reduz HP e ATK” sem número. Ficou metade do ATK e 10% do HP máximo por turno. Veneno “progressivo” também sem detalhe: percentual do HP máximo subindo a cada turno (5%, 10%, 15%...). Paralisia corta a SPD pela metade.

Fora isso, o guia do projeto não fala de stats dos iniciais, fórmula de dano, empate de SPD, limite de turno, se pode empatar, se status acumula, quanto a Potion cura, o que acontece se o HP passar de zero ou do máximo, nem o que fazer com nome fora da lista.

A fórmula a gente fechou só para o golpe ter um número. “Um status por vez” e “lutar até o fim” a gente escolheu de proposito, porque o enunciado não cobre.

Quase-contradição: o texto manda ordenar **estritamente** por SPD e não fala de empate. SPD igual precisa de alguma regra. Caso haja empate de SPD, começa quem tem mais ATK; se o ATK empatar também, o treinador 1 começa.

3 Requisitos autorais

Três regras nossas, já no sistema e com testes feitos. **não entra** aqui.

3.1 RA01 – Combo

O mesmo golpe duas vezes seguidas: o segundo ganha +50%. Tem golpe elemental (tipo + terreno) e Investida (neutra). Trocar de golpe ou usar item zera o combo. A Investida existe também para dar para testar o reset: se só tivesse um golpe, não tinha como mostrar “o outro reseta”.

3.2 RA02 – Lutar até o fim

A luta só acaba quando um cai. Sem teto de turno e sem empate. O caso chato é os dois zerarem no mesmo tick de status. Aí a gente olha o HP de cada um logo antes desse tick: ganha quem tinha mais vida. Se era igual, o treinador 1 vence.

3.3 RA03 – Um status por vez

Não junta Queimado + Veneno + Paralisado. O novo troca o antigo. Antidote limpa. Se o veneno entra no lugar da queimadura, o contador do veneno volta para zero. O enunciado lista os três e não diz se somam; somar deixava o fim de turno uma bagunça.

4 Modelagem UML

O sistema é todo feito no console (CLI), 1 contra 1, um inicial para cada um, um terreno.

4.1 Diagrama de casos de uso

O ator é o treinador (os dois fazem as mesmas coisas): escolher inicial, escolher terreno, golpe elemental, Investida, usar item e encerrar a batalha quando alguém cai.

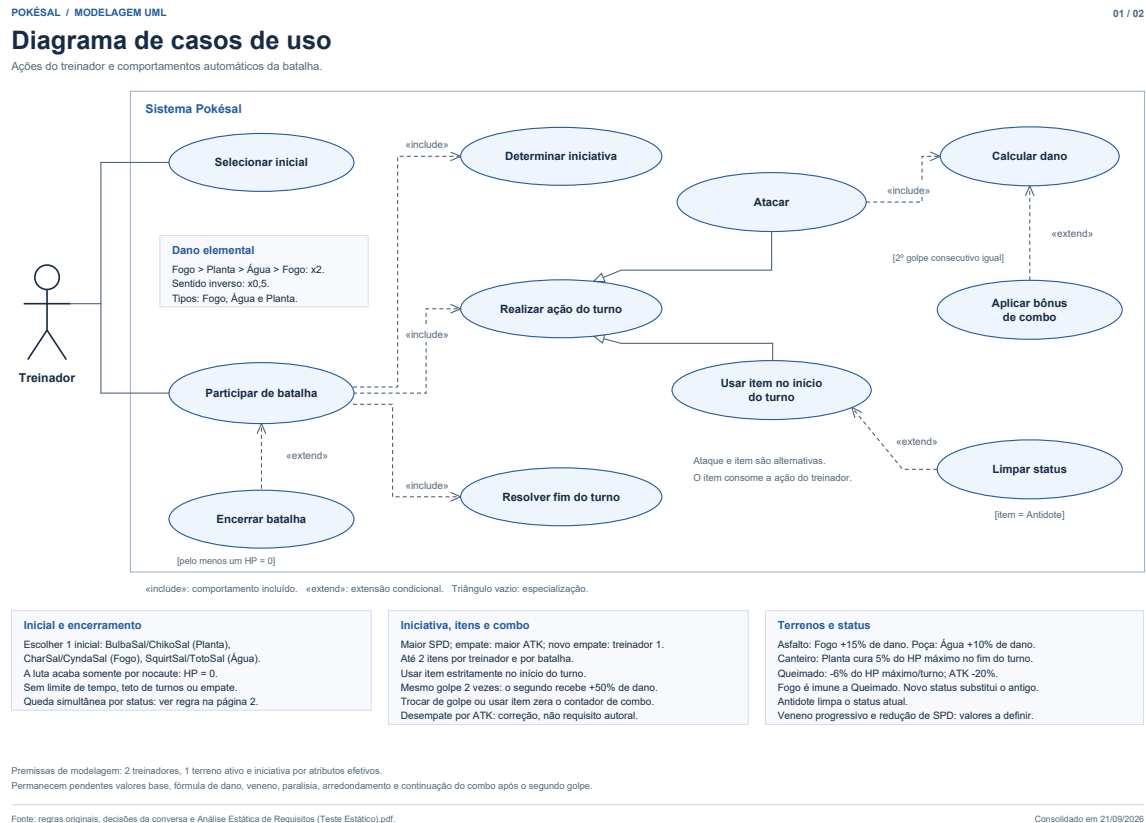


Figura 1 – Diagrama de casos de uso do PokeSal.

Inicial fora da lista não entra. Terceiro item também não. Encerrar batalha não é botão do menu: acontece quando o HP chega a zero, e sempre tem vencedor (RA02).

4.2 Diagrama de classes

Domínio em `pokesal.modelo`, luta em `pokesal.batalha`, exceção em `pokesal.excecao`. O Main só lê o que o usuário digitou e manda a batalha resolver o turno.

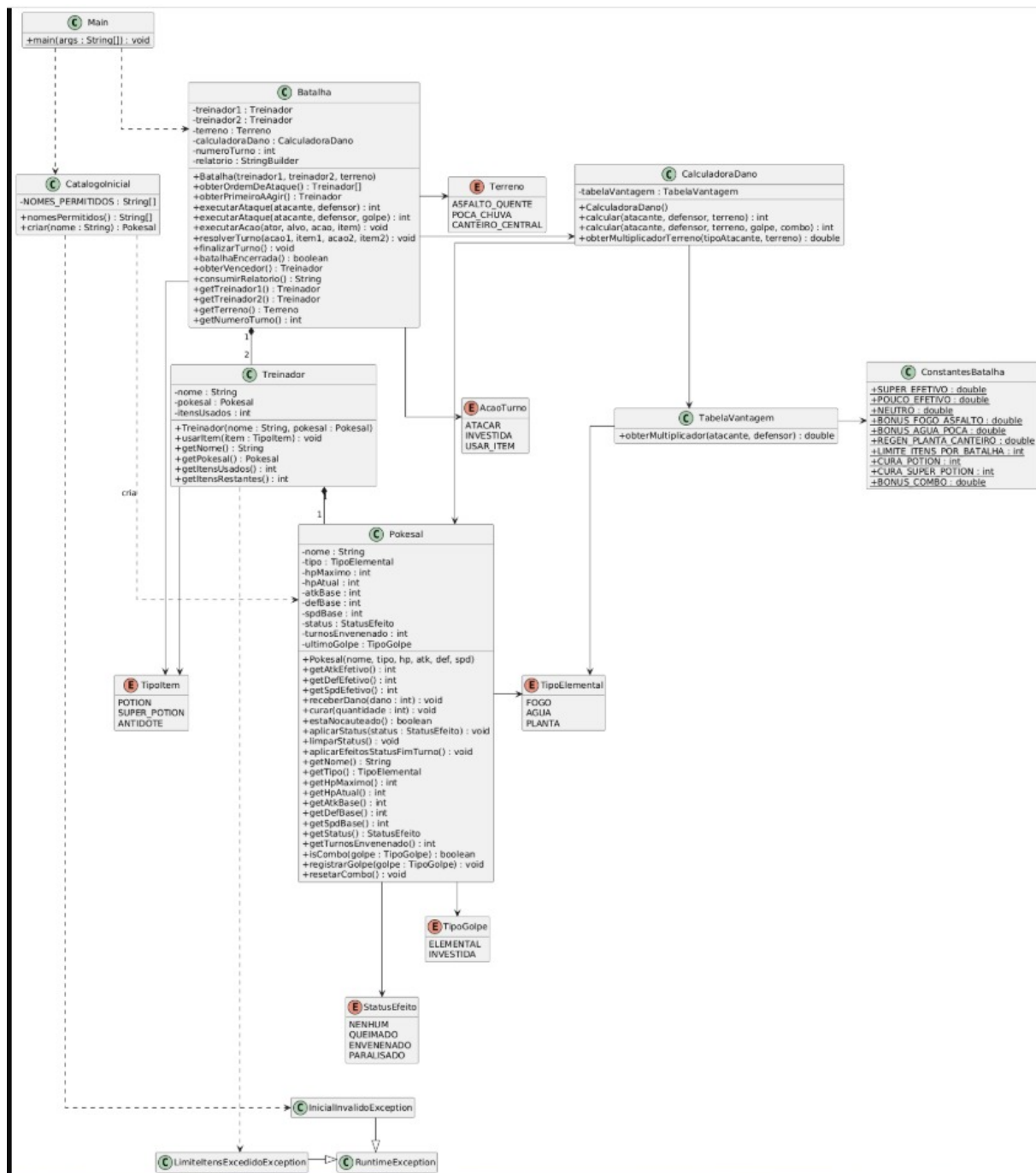


Figura 2 – Diagrama de classes do PokeSal.

Batalha concentra os dois treinadores, o terreno e o turno. Treinador tem um Pokesal e conta item. Pokesal segura HP, status e o combo (ultimoGolpe). CalculadoraDano usa TabelaVantagem e ConstantesBatalha. CatalogoInicial é quem cria os seis iniciais. Nome inválido vira InicialInvalidoException; terceiro item vira LimiteItensExcedidoException — as duas herdam RuntimeException.

5 Implementação e boas práticas (Checkstyle)

O código está no repositório Git, nos pacotes `pokesal`, `modelo`, `batalha` e `excecao`. A matéria pede Google Java Style. O que a gente aplicou:

- classe em PascalCase, método/variável em camelCase, constante em UPPER_SNAKE;
- Javadoc em classe e método público;
- número fixo de dano, vantagem, item e status em ConstantesBatalha;
- linha por volta de 100 caracteres e indentação de 2 espaços;
- import um a um, sem importar o pacote inteiro ou incluir WildCards.

O console pergunta treinador, inicial e terreno, e roda turno até ter vencedor.

6 Problemas encontrados e correções

Isso aqui apareceu quando a gente foi montar e testar no console:

Null na hora de criar a luta. Dava para subir batalha sem treinador, sem Pokésal ou sem terreno. Quebrava no meio do turno, com erro genérico. Agora isso é recusado na entrada.

HP negativo e cura sem teto. Golpe forte ia para baixo de zero; Potion com a vida cheia passava do máximo. HP para em zero; cura para no máximo da espécie.

Inicial inventado. Qualquer nome passava. Agora só os seis; o resto cai numa exceção só para isso.

Terceiro item. O limite era dois, mas a terceira tentativa não falhava de um jeito que o teste pegasse. A gente passou a lançar a exceção de limite e, no console, a pessoa escolhe outra ação em vez de o programa morrer.

SPD igual. Os dois com a mesma velocidade: a ordem mudava sem regra. Como o enunciado não cobre, desempate por ATK (não é autoral).

Os dois caíam no status. Dois queimados, pouca vida, o tick zerava os dois e não tinha vencedor. A gente guarda o HP de antes do tick e usa isso para decidir.

Combo depois do item. O +50% seguia mesmo depois de usar item. Item gasta o turno, então o combo zera.

Status empilhado. Dava para estar queimado e envenenado juntos, e o fim do turno cobrava os dois. Virou o RA03: o novo troca o antigo.

Número solto e import de pacote. Percentual escrito na conta e import com asterisco. Foi para constante nomeada e import explícito, que é o que o Checkstyle pede.

Se a gente entregasse assim, isso ia aparecer na arguição. Por isso corrigimos ainda nesta fase.

7 Relatório de contribuição individual

A divisão saiu na reunião de 18/09/2026, 18h00–20h30, no Google Meet. A segunda foi em 20/09/2026, 21h00–00h00: fechar código, revisar autoral, corrigir o que estava quebrando e organizar o Git. Os quatro estavam nas duas.

7.1 Guilherme Monteiro Da Silva

A inspeção de requisitos foi de Guilherme Monteiro. Ele leu o enunciado antes da gente programar e marcou detalhes importantes, como a observação da poça de chuva deixar em aberto “precisão ou dano”, falta de fórmula, empate de SPD, status sem número e o fato de o texto não dizer como o status entra na luta. Isso virou a Seção 2.

7.2 Guilherme Gomes de Magalhães

Os três autorais são de Guilherme Gomes: Combo, lutar até o fim e um status por vez. Também ajudou a deixar claro que desempate de SPD por ATK não é um requisito autoral, e sim correção do enunciado. O texto da Seção 3 é desse trabalho.

7.3 Miguel Novaes Palma Santos

Os diagramas UML são de Miguel: casos de uso e classes, batendo com o que o PokeSal faz de verdade (inicial, terreno, golpe, item, encerrar; Treinador, Pokesal, Batalha, CalculadoraDano, catálogo, enum e exceção).

7.4 Dantton Lavigne

Quem cuidou da maior parte do código foi Dantton: pacotes, interface no console, constante nomeada, Javadoc, indentação, imports e o ambiente para o resto da equipe rodar testes e apontar erros e acertos. Realizou também a primeira leva de testes com os nomes da Fase 02 e a conferência do Checkstyle.

7.5 O que a gente fez junto

Os quatro rodaram o projeto, bateram os autorais com o que estava funcionando, corrigiram o que está na Seção 6 e conferiram o que ia para o repositório.

8 Uso de inteligência artificial

A gente usou IA em duas frentes nesta fase: passar o olho na ortografia deste relatório e entender erro que o compilador apontava. Não foi a ferramenta escrevendo o trabalho no nosso lugar. A

gente redigia, compilava, e quando o texto ou o build quebrava pedia ajuda para achar o que estava errado.

No relatório, o que mais aparecia era acento, concordância e frase pela metade depois de um recorte. Algumas sugestão apareciam, a gente lia e aceitava ou não.

No compilador, dois tipos de erro repetiram. No Java, o Ctrl+F5 caía em `ClassNotFoundException` no Main: o programa procurava a classe em `target/classes` e a pasta estava vazia, porque o código ainda não tinha sido compilado nessa pasta. No $\text{\LaTeX}$, o que mais atrapalhou foi porcentagem solta no texto (o compilador trata % como comentário e come o resto da linha) e nome de classe com underline fora de `texttt`, que o `pdflatex` reclama. A IA explicava a mensagem; quem mexia no arquivo éramos nós.

Esses usos estão documentados no arquivo `AI_DECLARATION.md`, na raiz do repositório, junto com os prompts que a gente usou.

9 Fase 02 – Qualidade, testes e SonarQube

Essa parte é o que o enunciado da Fase 02 pede: checklist estático, JUnit, SonarQube e quem fez o quê. O código da batalha não mudou de regra; o que mudou foi teste, cobertura e uns avisos que o Sonar apontou.

9.1 Checklist de teste estático (revisão manual)

Guilherme Monteiro leu o código no Eclipse. Dantton passou isso para o arquivo `relatorios/checklist_`
Resumo do que a gente marcou:

1. Variáveis inicializadas antes do uso: sim. Local sem valor o Eclipse nem compila; campo entra com valor no construtor.
2. Variável declarada e nunca usada: não achamos. O Sonar também não marcou `unused`.
3. Código inacessível: não. O `while (true)` do menu tem `return` quando a opção vale.
4. Código duplicado: tinha. Nome dos iniciais escrito três vezes no `CatalogoInicial`. Virou constante. A cópia do array passou para `Arrays.copyOf`. Duplicação no Sonar ficou 0%.
5. Erro de sintaxe: não. Compila no Eclipse e no `mvn test`.
6. Erro de lógica de negócio: o jogo não estava calculando tipo/terreno errado. O Sonar achou um `if` redundante em `obterVencedor` (depois dos testes, `nocaute2` já era verdadeiro). A regra de não empatar continuou a mesma; a gente só limpou o `if`.
7. Erro de tipagem: não. Tinha um `cast (double)` `def` à toa na conta de dano; tiramos.

8. Fluxo de controle: o loop da luta não tem teto de turno de propósito (RA02). Alguém zera o HP. Menu pede de novo se a pessoa digita lixo.
9. Outros: o Sonar queria logger no lugar de System.out no Main. Não trocamos. É console, o jogador tem que ver o texto. Ficou como code smell aceito.

9.2 Testes JUnit e matriz de rastreabilidade

Os testes obrigatórios estão em `RegrasObrigatoriasTest`, com o nome que o PDF da matéria manda. Os autorais estão em `RequisitosAutoraisTest`. Tem um arquivo a mais, `CoberturaRegrasTest`, para catálogo, item e Investida, senão o Sonar ficava com coverage baixo.

Rodamos com `mvn test`. Passou. Um caso do RA02 (`testSemLimiteDeTurnos`) falhou na primeira: dois Pokésal de Planta caíam em menos de dez turnos, então o teste achava que existia teto. Não era bug do jogo; o cenário de HP/ATK estava curto demais. Ajustamos o cenário e passou.

Tabela 1 – Matriz de rastreabilidade: requisito $\times$ teste JUnit.

Requisito	Método	Resultado
Vantagem elemental	<code>testVantagemElemental</code>	passou
Terreno do estacionamento	<code>testEfeitoTerrenoEstacionamentoUCSal</code>	passou
Ordem por SPD	<code>testOrdemDeAtaquePorVelocidade</code>	passou
Limite de 2 itens	<code>testUsoLimiteDeItensExcedido</code>	passou
Valores-limite HP/ATK/DEF	<code>testCalculoDanoBoundaryValues</code>	passou
Empate de SPD (correção, não autoral)	<code>testEmpateSpdDesempatePorAtaque</code>	passou
RA01 Combo	<code>testComboGolpesIguaisAumentaDano,</code> <code>testComboResetaComOutroGolpeOuItem</code>	passou
RA02 Lutar até o fim	<code>testLutarAteOFimNaoEmpata,</code> <code>testSemLimiteDeTurnos</code>	passou
RA03 Um status por vez	<code>testStatusNaoAcumulaSubstituiAnterior</code>	passou
Catálogo, item, Investida	<code>CoberturaRegrasTest</code>	passou

9.3 SonarQube

Miguel rodou o sonar-scanner no servidor local (`localhost:9000`, projeto `pokesal-tqs`). Na primeira análise o Quality Gate passou, mas o coverage estava 0% — ainda não tinha Ja-CoCo. Maintainability (code smell) estava em 46. Security (vulnerabilities) e Reliability (bugs) já estavam em 0.

Depois dos testes e das correções, ele rodou de novo. As quatro métricas que o enunciado pede, no código geral (Overall Code):

- Bugs (Reliability): 0, nota A;

- Vulnerabilities (Security): 0, nota A;
- Code smells (Maintainability): 36 abertos, nota A — quase tudo System.out do console, que a gente não trocou por logger;
- Coverage: 84,3% em 325 linhas (o Main de menu ficou de fora da cobertura, porque é entrada no teclado).

No recorte de código novo (New Code) o coverage foi 90,5%. Duplicação 0%. Quality Gate: Passed. Os prints estão nas Figuras 3 e 4.

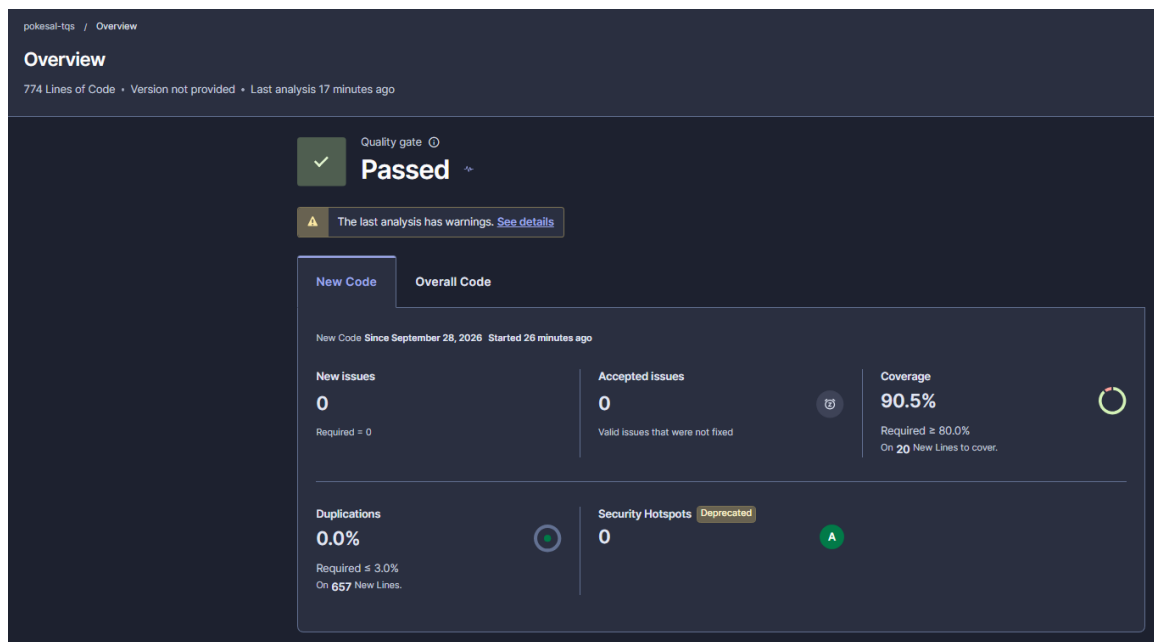


Figura 3 – SonarQube – New Code: Quality Gate Passed e coverage 90,5%.

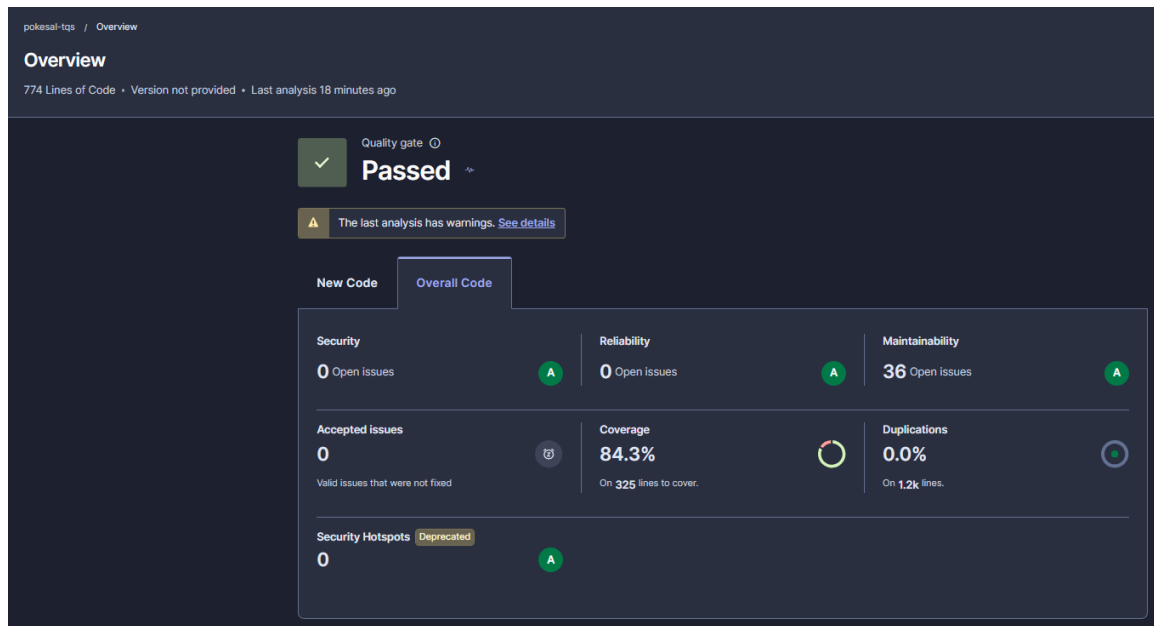


Figura 4 – SonarQube – Overall Code: Security 0, Reliability 0, Maintainability 36, coverage 84,3%.

9.4 Bugs da Fase 02 e o que a gente corrigiu

Isso aqui saiu do estático, do JUnit e do Sonar (não é o mesmo da Seção 6, que foi montar o jogo na Fase 01):

Literal repetido. BulbaSal e o resto dos nomes, três vezes no catálogo. Constante.

Cópia de array na mão. nomesPermitidos fazia for. Passou a `Arrays.copyOf`.

Condição sempre verdadeira. obterVencedor: depois de testar “só o 1 caiu”, o `nocaute2` no próximo `if` já era verdade. A regra (não empata; se os dois caem, HP de antes do status) não mudou. O `if` que mudou.

Cast inútil. (`double`) `def` na fórmula. Saiu.

`isCombo` **com vários if.** Virou um `return`.

Coverage 0%. Não era bug de regra. Faltava JUnit + JaCoCo no `pom.xml` e caminho do XML no `sonar-project.properties`.

Teste `testSemLimiteDeTurnos` **vermelho.** Cenário fraco, não o código da batalha.

O que **não** corrigimos: `System.out` no `Main`. É saída do jogo.

9.5 Contribuição individual da Fase 02

Dantton escreveu os testes JUnit (obrigatórios, autorais e o de cobertura) e depois alterou o código no que o Sonar e a revisão apontaram (constante, obterVencedor, cast, isCombo, JaCoCo). Também fechou o checklist em txt.

Miguel rodou a primeira análise no SonarQube, viu o coverage zerado e os code smell, e depois refez o scan quando o código e os testes já estavam no lugar. Os prints desta seção são dessa segunda passada.

Guilherme Monteiro apontou o que valia corrigir (o High dos nomes, o if do vencedor, o que era só aviso de console) e fez a revisão manual do código, que virou o checklist.

Guilherme Gomes revisou o pacote de testes e, no fim, os números do Sonar (Quality Gate, as quatro métricas e o que ficou aberto de propósito).

10 Considerações finais

A Fase 01 termina com o jogo rodando, as regras do enunciado, três autorais, UML batendo com o código e fonte no padrão de nome, Javadoc, constantes e imports.

O enunciado tem alguns buracos. A gente marcou, separou o que era correção e o que era autoral, e consertou o que só aparece quando o programa roda (null, HP inválido, item a mais, combo que não zerava, os dois caindo no mesmo tick).

A Fase 02 fecha o que a matéria pedia de qualidade: checklist estático (Seção 9.1), JUnit com matriz (Seção 9.2), SonarQube com bugs, vulnerabilidades, code smell e coverage (Seção 9.3) e a divisão de trabalho dessa etapa (Seção 9.5). Quality Gate passou. O que ficou aberto no Sonar é o `System.out` do console.